

Nonlinear Regression

Synopsis

Common approaches to solving nonlinear regression problems are introduced, extending the development of linear regression in Chapter 2. We begin with a discussion of Newton's method, which provides a general framework for solving nonlinear systems of equations and nonlinear optimization problems. Then we discuss the Gauss-Newton (GN) and Levenberg-Marquardt (LM) methods, which are versions of Newton's method specialized for nonlinear regression problems. The distinction between LM and Tikhonov regularization is also made. Statistical aspects and implementation issues are addressed, and examples of nonlinear regression are presented.

9.1. INTRODUCTION TO NONLINEAR REGRESSION

In previous chapters we have concentrated on linear forward and inverse problems, and have seen that such problems are uniformly approachable using a variety of solution methods. We will next consider problems that are nonlinear, i.e., forward and inverse problems that do not obey the rules of superposition (1.5) and scaling (1.6). There is no general theory for the solution of nonlinear parameter estimation and inverse problems. However, we will see that iterative strategies incorporating linear concepts can be applied to solve them in many circumstances.

9.2. NEWTON'S METHOD FOR SOLVING NONLINEAR EQUATIONS

Consider a nonlinear system of m equations in m unknowns

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}. \quad (9.1)$$

We will construct a sequence of vectors, $\mathbf{x}^0, \mathbf{x}^1, \dots$, that will converge to a solution $\mathbf{x}^*$. If the nonlinear vector function $\mathbf{F}$ is continuously differentiable, we can construct a Taylor series approximation about some starting solution estimate, $\mathbf{x}^0$,

$$\mathbf{F}(\mathbf{x}^0 + \Delta\mathbf{x}) \approx \mathbf{F}(\mathbf{x}^0) + \mathbf{J}(\mathbf{x}^0)\Delta\mathbf{x}, \quad (9.2)$$

where $\mathbf{J}(\mathbf{x}^0)$ is the Jacobian,

$$\mathbf{J}(\mathbf{x}^0) = \left[\begin{array}{ccc} \frac{\partial F_1(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial F_1(\mathbf{x})}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_m(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial F_m(\mathbf{x})}{\partial x_m} \end{array} \right] \bigg|_{\mathbf{x}=\mathbf{x}^0}. \quad (9.3)$$

Using (9.2), and expressing the difference between the desired solution, $\mathbf{x}^*$, and $\mathbf{x}^0$ as

$$\Delta \mathbf{x} = \mathbf{x}^* - \mathbf{x}^0 \quad (9.4)$$

gives

$$\mathbf{F}(\mathbf{x}^*) = \mathbf{0} \approx \mathbf{F}(\mathbf{x}^0) + \mathbf{J}(\mathbf{x}^0)\Delta \mathbf{x}, \quad (9.5)$$

which produces a linear system of equations,

$$\mathbf{J}(\mathbf{x}^0)\Delta \mathbf{x} \approx -\mathbf{F}(\mathbf{x}^0), \quad (9.6)$$

that can be solved for $\Delta \mathbf{x}$ to improve the solution estimate.

Algorithm 9.1 Newton's Method

Given a system of equations $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ and an initial solution $\mathbf{x}^0$, repeat the following steps to compute a sequence of solutions $\mathbf{x}^k$. Stop if and when the sequence adequately converges to a solution with $\mathbf{F}(\mathbf{x}) = \mathbf{0}$.

1. Calculate the Jacobian $\mathbf{J}(\mathbf{x}^k)$ and $\mathbf{F}(\mathbf{x}^k)$.
 2. Solve $\mathbf{J}(\mathbf{x}^k)\Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}^k)$.
 3. Let $\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta \mathbf{x}$.
 4. Let $k = k + 1$.
-

The theoretical properties of Newton's method are summarized in the following theorem. For a proof, see [39].

Theorem 9.1 *If $\mathbf{x}^0$ is close enough to $\mathbf{x}^*$, $\mathbf{F}(\mathbf{x})$ is continuously differentiable in a neighborhood of $\mathbf{x}^*$, and $\mathbf{J}(\mathbf{x}^*)$ is nonsingular, then Newton's method will converge to $\mathbf{x}^*$. The convergence rate is quadratic in the sense that there is a constant c , such that for large k ,*

$$\|\mathbf{x}^{k+1} - \mathbf{x}^*\|_2 \leq c \|\mathbf{x}^k - \mathbf{x}^*\|_2^2. \quad (9.7)$$

In practical terms, quadratic convergence means that as we approach $\mathbf{x}^*$, the number of accurate digits in the solution doubles at each iteration. Unfortunately, if the hypotheses in the above theorem are not satisfied, then Newton's method can converge very slowly or even fail altogether.

A simple modification to Newton's method often helps with convergence problems. In the **damped Newton's method**, we use the Newton's method equations at each iteration to compute an iterative correction to the working solution. However, instead of taking the full step $\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta\mathbf{x}$, we perform a **line search** across solutions lying along the line $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha\Delta\mathbf{x}$ for a range of positive α values and take the one with minimum $\|\mathbf{F}(\mathbf{x}^{k+1})\|_2$.

Now suppose that we wish to minimize a scalar-valued function $f(\mathbf{x})$. If we assume that $f(\mathbf{x})$ is twice continuously differentiable, we can construct a Taylor series approximation,

$$f(\mathbf{x}^0 + \Delta\mathbf{x}) \approx f(\mathbf{x}^0) + \nabla f(\mathbf{x}^0)^T \Delta\mathbf{x} + \frac{1}{2} \Delta\mathbf{x}^T \mathbf{H}(f(\mathbf{x}^0)) \Delta\mathbf{x}, \quad (9.8)$$

where $\nabla f(\mathbf{x}^0)$ is the gradient

$$\nabla f(\mathbf{x}^0) = \left[\begin{array}{c} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_m} \end{array} \right] \bigg|_{\mathbf{x}=\mathbf{x}^0} \quad (9.9)$$

and $\mathbf{H}(f(\mathbf{x}^0))$ is the Hessian

$$\mathbf{H}(f(\mathbf{x}^0)) = \left[\begin{array}{ccc} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_m \partial x_1} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_m^2} \end{array} \right] \bigg|_{\mathbf{x}=\mathbf{x}^0}. \quad (9.10)$$

A necessary condition for $\mathbf{x}^*$ to be a minimum of $f(\mathbf{x})$ is that $\nabla f(\mathbf{x}^*) = \mathbf{0}$. We can approximate the gradient in the vicinity of $\mathbf{x}^0$ by

$$\nabla f(\mathbf{x}^0 + \Delta\mathbf{x}) \approx \nabla f(\mathbf{x}^0) + \mathbf{H}(f(\mathbf{x}^0)) \Delta\mathbf{x}. \quad (9.11)$$

Setting the approximate gradient (9.11) equal to zero gives

$$\mathbf{H}(f(\mathbf{x}^0)) \Delta\mathbf{x} = -\nabla f(\mathbf{x}^0). \quad (9.12)$$

Solving the linear system of equations (9.12) for successive solution steps leads to **Newton's method for minimizing $f(\mathbf{x})$** .

Since Newton's method for minimizing $f(\mathbf{x})$ is exactly Newton's method for solving a nonlinear system of equations applied to $\mathbf{F} = \nabla f(\mathbf{x}) = \mathbf{0}$, the convergence proof follows immediately from the proof of Theorem 9.1.

Theorem 9.2 *If $f(\mathbf{x})$ is twice continuously differentiable in a neighborhood of a local minimizer $\mathbf{x}^*$, there is a constant λ such that $\|\mathbf{H}(f(\mathbf{x})) - \mathbf{H}(f(\mathbf{y}))\|_2 \leq \lambda \|\mathbf{x} - \mathbf{y}\|_2$ for every*

vector $\mathbf{y}$ in the neighborhood, $\mathbf{H}(f(\mathbf{x}^*))$ is positive definite, and $\mathbf{x}^0$ is close enough to $\mathbf{x}^*$, then Newton's method will converge quadratically to $\mathbf{x}^*$.

Algorithm 9.2 Newton's Method for Minimizing $f(\mathbf{x})$

Given a twice continuously differentiable function $f(\mathbf{x})$, and an initial solution $\mathbf{x}^0$, repeat the following steps to compute a sequence of solutions $\mathbf{x}^1, \mathbf{x}^2, \dots$. Stop if and when the sequence adequately converges to a solution with $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

1. Calculate the gradient $\nabla f(\mathbf{x}^k)$ and Hessian $\mathbf{H}(f(\mathbf{x}^k))$.
 2. Solve $\mathbf{H}(f(\mathbf{x}^k))\Delta\mathbf{x} = -\nabla f(\mathbf{x}^k)$.
 3. Let $\mathbf{x}^{k+1} = \mathbf{x}^k + \Delta\mathbf{x}$.
 4. Let $k = k + 1$.
-

Newton's method for minimizing $f(\mathbf{x})$ can be very efficient, but the method can also fail to converge. As with Newton's method for solving a system of equations, the convergence properties of the algorithm can be improved in practice by modifying the model update step with a line search.

9.3. THE GAUSS-NEWTON AND LEVENBERG-MARQUARDT METHODS FOR SOLVING NONLINEAR LEAST SQUARES PROBLEMS

Newton's method is not directly applicable to most nonlinear regression and inverse problems. We may not have equal numbers of data points and model parameters, there may not be an exact solution to $\mathbf{G}(\mathbf{m}) = \mathbf{d}$, or $\mathbf{G}(\mathbf{m}) = \mathbf{d}$ may have multiple solutions. Here, we will use a specialized version of Newton's method to minimize a nonlinear least squares problem.

Given a nonlinear system of equations $\mathbf{G}(\mathbf{m}) = \mathbf{d}$, consider the problem of finding an n -length parameter vector, $\mathbf{m}$, constrained by an n -length data vector, $\mathbf{d}$, with associated specified data standard deviations. Our goal is to find a set of parameters that best fits the data in the sense of minimizing the 2-norm of the residuals.

As with linear regression, if we assume that the measurement errors are normally distributed, then the maximum likelihood principle leads us to minimizing the sum of squared residuals normalized by their respective standard deviations (2.12). We seek to minimize the weighted residual norm

$$f(\mathbf{m}) = \sum_{i=1}^m \left(\frac{G(\mathbf{m})_i - d_i}{\sigma_i} \right)^2. \quad (9.13)$$

We define the scalar-valued functions

$$f_i(\mathbf{m}) = \frac{G(\mathbf{m})_i - d_i}{\sigma_i} \quad i = 1, 2, \dots, m \quad (9.14)$$

and the vector-valued function

$$\mathbf{F}(\mathbf{m}) = \begin{bmatrix} f_1(\mathbf{m}) \\ \vdots \\ f_m(\mathbf{m}) \end{bmatrix}. \quad (9.15)$$

Thus

$$f(\mathbf{m}) = \sum_{i=1}^m f_i(\mathbf{m})^2 = \|\mathbf{F}(\mathbf{m})\|_2^2. \quad (9.16)$$

The gradient of $f(\mathbf{m})$ can be written as the sum of the gradients of the individual terms,

$$\nabla f(\mathbf{m}) = \sum_{i=1}^m \nabla (f_i(\mathbf{m})^2). \quad (9.17)$$

The elements of the gradient of $f(\mathbf{m})$ are

$$(\nabla f(\mathbf{m}))_j = \sum_{i=1}^m 2f_i(\mathbf{m})(\nabla f_i(\mathbf{m}))_j, \quad (9.18)$$

and the gradient can thus be written in matrix notation as

$$\nabla f(\mathbf{m}) = 2\mathbf{J}(\mathbf{m})^T \mathbf{F}(\mathbf{m}), \quad (9.19)$$

where $\mathbf{J}(\mathbf{m})$ is the Jacobian

$$\mathbf{J}(\mathbf{m}) = \begin{bmatrix} \frac{\partial f_1(\mathbf{m})}{\partial m_1} & \cdots & \frac{\partial f_1(\mathbf{m})}{\partial m_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m(\mathbf{m})}{\partial m_1} & \cdots & \frac{\partial f_m(\mathbf{m})}{\partial m_n} \end{bmatrix}. \quad (9.20)$$

Similarly, we can express the Hessian of $f(\mathbf{m})$ using the $f_i(\mathbf{m})$ terms to obtain

$$\mathbf{H}(f(\mathbf{m})) = \sum_{i=1}^m \mathbf{H}(f_i(\mathbf{m})^2) \quad (9.21)$$

$$= \sum_{i=1}^m \mathbf{H}^i(\mathbf{m}), \quad (9.22)$$

where $\mathbf{H}^i(\mathbf{m})$ is the Hessian of $f_i(\mathbf{m})^2$.

The j, k element of $\mathbf{H}^i(\mathbf{m})$ is

$$H_{j,k}^i(\mathbf{m}) = \frac{\partial^2 (f_i(\mathbf{m})^2)}{\partial m_j \partial m_k} \quad (9.23)$$

$$= \frac{\partial}{\partial m_j} \left(2f_i(\mathbf{m}) \frac{\partial f_i(\mathbf{m})}{\partial m_k} \right) \quad (9.24)$$

$$= 2 \left(\frac{\partial f_i(\mathbf{m})}{\partial m_j} \frac{\partial f_i(\mathbf{m})}{\partial m_k} + f_i(\mathbf{m}) \frac{\partial^2 f_i(\mathbf{m})}{\partial m_j \partial m_k} \right). \quad (9.25)$$

Thus

$$\mathbf{H}(f(\mathbf{m})) = 2\mathbf{J}(\mathbf{m})^T \mathbf{J}(\mathbf{m}) + \mathbf{Q}(\mathbf{m}), \quad (9.26)$$

where

$$\mathbf{Q}(\mathbf{m}) = 2 \sum_{i=1}^m f_i(\mathbf{m}) \mathbf{H}(f_i(\mathbf{m})). \quad (9.27)$$

In the **Gauss-Newton (GN) method**, we ignore the $\mathbf{Q}(\mathbf{m})$ term in (9.26) and approximate the Hessian as

$$\mathbf{H}(f(\mathbf{m})) \approx 2\mathbf{J}(\mathbf{m})^T \mathbf{J}(\mathbf{m}). \quad (9.28)$$

In the context of nonlinear regression, we expect that the $f_i(\mathbf{m})$ terms will be small as we approach the optimal parameters $\mathbf{m}^*$, so that this should be a reasonable approximation in the vicinity of the solution. Conversely, this is not a reasonable approximation for nonlinear least squares problems in which the values of $f_i(\mathbf{m})$ can be large.

Implementing Newton's method for minimizing $f(\mathbf{m})$ (9.12) using the gradient (9.19) and the approximate Hessian (9.28), and dividing both sides by 2, we obtain

$$\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k) \Delta \mathbf{m} = -\mathbf{J}(\mathbf{m}^k)^T \mathbf{F}(\mathbf{m}^k), \quad (9.29)$$

which provides a formula for solving for successive update steps $\Delta \mathbf{m}$. The n by n matrix $\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k)$ is symmetric and positive semidefinite. If the matrix is actually positive definite then we can use the Cholesky factorization or another full-rank method to solve the system of equations for $\Delta \mathbf{m}$. However, if the matrix is singular then such straightforward approaches will fail. Although the GN method often works well in practice, it is based on Newton's method, and can thus fail by converging to a local maximum or saddle point where $\nabla f(\mathbf{m}) \approx 0$, or by not converging at all. An additional point to consider in using GN and other such iterative methods is that the algorithm may converge to a local minimum rather than to a global minimum (see Section 9.5).

In the **Levenberg-Marquardt (LM) method**, the GN method model update equations (9.29) are modified to

$$(\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k) + \lambda \mathbf{I}) \Delta \mathbf{m} = -\mathbf{J}(\mathbf{m}^k)^T \mathbf{F}(\mathbf{m}^k), \quad (9.30)$$

and the positive parameter λ is adjusted during the course of iterations to ensure convergence. One important reason for this modification is to ensure that the matrix on the left-hand side of (9.30) is nonsingular. Since the matrix in this system of equations is symmetric and positive definite, we can use the Cholesky factorization to efficiently solve the system for the model update steps $\delta \mathbf{m}$.

For very large values of λ ,

$$\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k) + \lambda \mathbf{I} \approx \lambda \mathbf{I}, \quad (9.31)$$

and the solution to (9.30) is

$$\Delta \mathbf{m} \approx -\frac{1}{\lambda} \nabla f(\mathbf{m}). \quad (9.32)$$

This is called a **steepest-descent** step, meaning that the algorithm simply moves down-gradient to most rapidly reduce $f(\mathbf{m})$. The steepest-descent approach provides very slow but certain convergence to a local minimum. Conversely, for very small values of λ , the LM method reverts to the GN method (9.29), which gives potentially fast but uncertain convergence.

A challenge associated with implementing the LM method is determining the optimal value of λ . The general strategy is to use small values of λ in situations where the GN method is working well, but to increase λ when the GN method fails to make progress in reducing the residual norm. A simple approach is to start with a small value of λ , and then adjust it in every iteration. If the LM method leads to a residual norm reduction, then update $\mathbf{m}$ and decrease λ by a constant multiplicative factor (e.g., a factor of 2) before the next iteration. Conversely, if the LM method does not improve the solution, we then increase λ by a constant factor and try again, repeating this process until a model update is found that decreases the residual norm. Robust implementations of the LM method use more sophisticated strategies for adjusting λ , but even this simple strategy works surprisingly well.

In practice, a careful LM implementation offers the good performance of the GN method as well as robust convergence properties, and LM is usually the method of choice for small- to medium-sized nonlinear least squares problems.

Note that although the LM stabilization term $\lambda \mathbf{I}$ in (9.30) resembles expressions used elsewhere (e.g., Chapter 4) for Tikhonov regularization purposes, it does not alter the ultimate model achieved at convergence. The $\lambda \mathbf{I}$ term is used to stabilize the solution of the linear system of equations that determines the search direction to be used. Because the $\lambda \mathbf{I}$ term is only used as a way to improve the convergence of the algorithm, and does *not* enter into the residual norm objective function that is being minimized, it does not regularize the nonlinear least squares problem. We discuss the regularization of nonlinear problems in Chapter 10.

9.4. STATISTICAL ASPECTS OF NONLINEAR LEAST SQUARES

Recall from Appendix B that if a vector $\mathbf{d}$ has a multivariate normal distribution, and $\mathbf{A}$ is an appropriately sized matrix, then $\mathbf{A}\mathbf{d}$ also has a multivariate normal distribution with an associated covariance matrix

$$\text{Cov}(\mathbf{A}\mathbf{d}) = \mathbf{A}\text{Cov}(\mathbf{d})\mathbf{A}^T. \quad (9.33)$$

In Chapter 2, we applied this formula to the linear least squares problem for $\mathbf{G}\mathbf{m} = \mathbf{d}$, which we solved by the normal equations. The resulting formula for $\text{Cov}(\mathbf{m})$ was

$$\text{Cov}(\mathbf{m}_{L_2}) = (\mathbf{G}^T \mathbf{G})^{-1} \mathbf{G}^T \text{Cov}(\mathbf{d}) \mathbf{G} (\mathbf{G}^T \mathbf{G})^{-1}. \quad (9.34)$$

In the simplest case, where $\text{Cov}(\mathbf{d}) = \sigma^2 \mathbf{I}$, (9.34) simplified to

$$\text{Cov}(\mathbf{m}_{L_2}) = \sigma^2 (\mathbf{G}^T \mathbf{G})^{-1}. \quad (9.35)$$

For the nonlinear regression problem, we no longer have a linear relationship between the data and the estimated model parameters, so we cannot assume that the estimated model parameters have a multivariate normal distribution, and cannot use the above formulas. However, we can obtain useful corresponding formulas by linearization if the data errors are not too large.

If small data perturbations result in small model perturbations in a nonlinear system, we can consider a linearization of the misfit function (9.15) about a solution, $\mathbf{m}^*$:

$$\mathbf{F}(\mathbf{m}^* + \Delta\mathbf{m}) \approx \mathbf{F}(\mathbf{m}^*) + \mathbf{J}(\mathbf{m}^*)\Delta\mathbf{m}. \quad (9.36)$$

Under this approximation, there is a linear relationship between changes in $\mathbf{F}$ and changes in the parameters $\mathbf{m}$:

$$\mathbf{F}(\mathbf{m}^* + \Delta\mathbf{m}) - \mathbf{F}(\mathbf{m}^*) = \Delta\mathbf{F} \approx \mathbf{J}(\mathbf{m}^*)\Delta\mathbf{m}. \quad (9.37)$$

To the extent that the residual misfit terms (the elements of $\mathbf{F}(\mathbf{m}^*)$) are small, the Hessian can be approximated by (9.28). In this case $\mathbf{J}(\mathbf{m}^*)$ in nonlinear regression can take the place of $\mathbf{G}$ in linear problems (e.g., (9.35)) to estimate the covariance of the model parameters. Equation (9.13) incorporates the respective data element standard deviations σ_i into the formula for the residual norm $f(\mathbf{m})$, and explicitly weights the nonlinear constraint equations. $\text{Cov}(\mathbf{d})$, in the case of independent data errors, is thus the identity matrix in such a weighted system. In this case we have

$$\text{Cov}(\mathbf{m}^*) \approx (\mathbf{J}(\mathbf{m}^*)^T \mathbf{J}(\mathbf{m}^*))^{-1}. \quad (9.38)$$

As in Chapter 2, we can construct a 95% confidence ellipsoid for the fitted parameters in terms of this covariance matrix,

$$(\mathbf{m} - \mathbf{m}^*)^T \mathbf{J}(\mathbf{m}^*)^T \mathbf{J}(\mathbf{m}^*) (\mathbf{m} - \mathbf{m}^*) \leq \Delta^2, \quad (9.39)$$

where Δ^2 is the ninety-fifth percentile of a χ^2 distribution with n degrees of freedom. It is important to reiterate that this confidence ellipsoid only defines an approximate confidence region because we linearized $\mathbf{F}(\mathbf{m})$ in (9.37).

As in Chapter 2, there is an alternative derivation of the 95% confidence region for the fitted parameters in terms of $\chi^2(\mathbf{m}) - \chi^2(\mathbf{m}^*)$. If we let

$$\chi^2(\mathbf{m}) = \|\mathbf{F}(\mathbf{m})\|_2^2 = \mathbf{F}(\mathbf{m})^T \mathbf{F}(\mathbf{m}), \quad (9.40)$$

then the inequality

$$\chi^2(\mathbf{m}) - \chi^2(\mathbf{m}^*) \leq \Delta^2 \quad (9.41)$$

defines an approximate 95% confidence region for $\mathbf{m}^*$. Because of the nonlinearity in the $\chi^2(\mathbf{m})$ function, this 95% confidence region is typically not an ellipsoid. Unfortunately, this confidence region is not exact, because this development is still based on the assumption of multivariate normality, and $\mathbf{m}^*$, because it is not a linear combination of the data vector elements, will not in general have a multivariate normal distribution in nonlinear problems.

Should (9.39) or (9.41) be used in practice? If $G(\mathbf{m})$ is not too strongly nonlinear, then there should be very little difference between the two confidence regions. However, if $G(\mathbf{m})$ is more strongly nonlinear, then the difference between the two confidence regions may be large and neither confidence region can really be trusted. In such situations, Monte Carlo methods are a more useful and appropriate way to estimate parameter uncertainty. This is discussed further in Chapter 11.

As with linear regression, it is possible to apply nonlinear regression when the measurement errors are independent and normally distributed and the standard deviations are unknown but assumed to be equal (Section 2.3). We set the σ_i to 1 and minimize the sum of squared errors, and define a residual vector,

$$r_i = G(\mathbf{m}^*)_i - d_i \quad i = 1, 2, \dots, m. \quad (9.42)$$

Our estimate of the measurement standard deviation is then

$$s = \sqrt{\frac{\|\mathbf{r}\|_2^2}{m - n}}, \quad (9.43)$$

and the corresponding approximate covariance matrix for the estimated model parameters is

$$\text{Cov}(\mathbf{m}^*) = s^2 (\mathbf{J}(\mathbf{m}^*)^T \mathbf{J}(\mathbf{m}^*))^{-1}. \quad (9.44)$$

Given $\mathbf{m}^*$ and $\text{Cov}(\mathbf{m}^*)$, we can establish confidence intervals for the model parameters using the methods described in Chapter 2. As with any parameter estimation problem, it is also important to examine the residuals for systematic patterns or deviations from normality. If we have not estimated the measurement standard deviation s from the fitted residuals, then it is also important to test the χ^2 value for goodness of fit and perform an associated **p-value** test (2.21) if we have normally distributed data errors. The appropriateness of this test will depend on how well the nonlinear model is approximated by the Jacobian linearization for points near the optimal parameter values.

Example 9.1

A classic method in hydrology for determining the transmissivity and storage coefficient of an aquifer is called the “slug test” [49].

A known volume Q of water (the slug) is injected into a well, and the resulting effects on the head, h , at an observation well a distance d away from the injection well are observed at various times t . The change in the head measured at the observation well typically increases rapidly and then decreases more slowly. We wish to determine the dimensionless storage coefficient, S , and the transmissivity T .

The mathematical model for the slug test is

$$h = \frac{Q}{4\pi Tt} e^{-d^2 S/(4Tt)}. \quad (9.45)$$

We know the parameters $Q = 50 \text{ m}^3$ and $d = 60 \text{ m}$, and the times t at which the head h is measured. Our data are given in Table 9.1, where head measurements are roughly accurate to 0.01 m ($\sigma_i = 0.01 \text{ m}$).

The optimal parameter values are $S = 0.00207$ and $T = 0.585 \text{ m}^2/\text{hr}$. The observed χ^2 value is 2.04, with a corresponding p -value of 0.73. Thus this fit passes the χ^2 test. The data points and fitted curve are shown in Figure 9.1.

Using the Jacobian evaluated at the optimal parameter values, we computed an approximate covariance matrix for the fitted parameters. The resulting 95% confidence intervals for S and T are

$$S = 0.00207 \pm 0.00012 \quad (9.46)$$

$$T = 0.585 \pm 0.029 \text{ m}^2/\text{hr}. \quad (9.47)$$

Table 9.1 Slug Test Data

t (hr)	5	10	20	30	40	50
h (m)	0.72	0.49	0.30	0.20	0.16	0.12



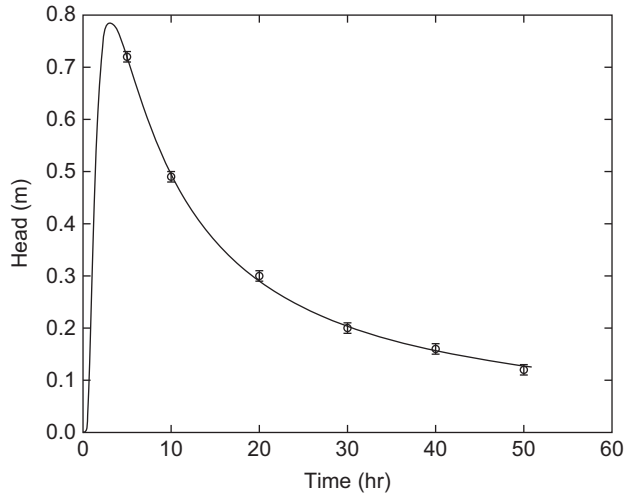


Figure 9.1 Data, with one standard deviation error bars, and fitted model for the slug test.

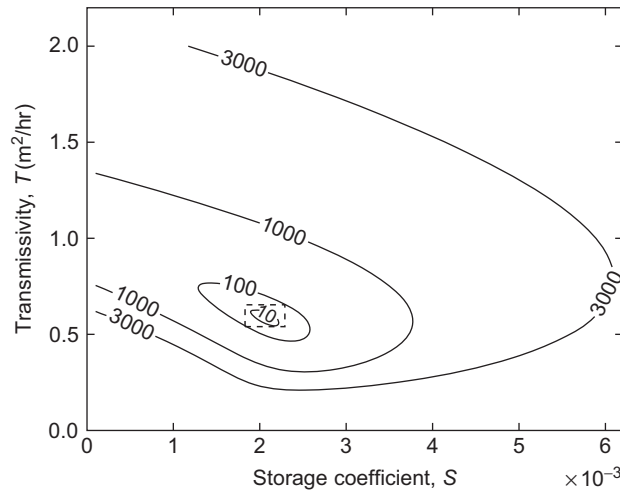


Figure 9.2 χ^2 contour plot for the slug test for a wide range of parameters. The small box shows the close-up axis range displayed in [Figure 9.3](#).

A contour plot of the χ^2 surface obtained by varying S and T is shown in [Figure 9.2](#). Note that, unlike our earlier linear regression problems, the contours are not even approximately elliptical because of the nonlinearity of the problem. However, if we zoom into the immediate vicinity of the optimal parameters ([Figure 9.3](#)), we find that the χ^2 contours are approximately elliptical at this scale. The approximate ellipticity

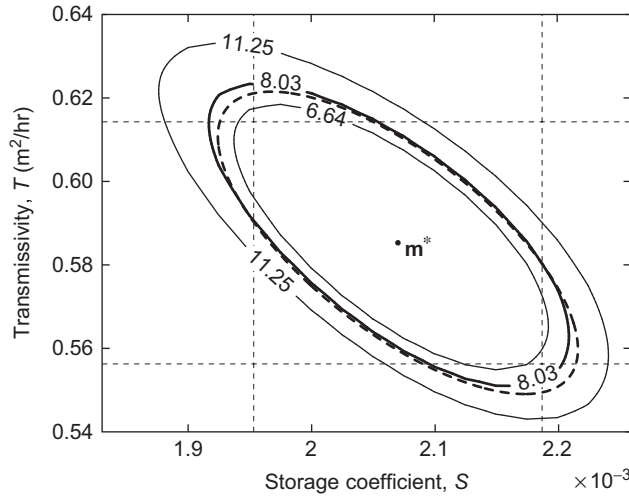


Figure 9.3 Close-up χ^2 contour plot for the immediate vicinity of the optimal parameters $\mathbf{m}^*$ for the slug test example, showing nonlinear 90%, 95% (bold), and 99% confidence contours. The nearly elliptical contours are indicative of an approximately linear behavior at this scale. Dashed ellipse shows the linearized approximation to the 95% confidence contour calculated using (9.38). Ninety-five percent confidence intervals for the parameters calculated from (9.38) are shown as dashed lines.

of the contours indicates that the linear approximation of $\mathbf{G}(\mathbf{m})$ around the optimal parameter values is a reasonable approximation for model perturbations in this range.

9.5. IMPLEMENTATION ISSUES

Iterative methods for solving nonlinear problems require the computation of the functions $f_i(\mathbf{m})$ and their partial derivatives with respect to the model parameters m_j . These partial derivatives in turn depend on the derivatives of the nonlinear $\mathbf{G}$:

$$\frac{\partial f_i(\mathbf{m})}{\partial m_j} = \frac{1}{\sigma_i} \frac{\partial \mathbf{G}(\mathbf{m})_i}{\partial m_j}. \quad (9.48)$$

In some cases, we have explicit formulas for $\mathbf{G}(\mathbf{m})$ and its derivatives. In other cases, $\mathbf{G}(\mathbf{m})$ exists only as a **black box** subroutine that can be called as required to evaluate the function.

When an explicit formula for $\mathbf{G}(\mathbf{m})$ is available, and the number of parameters is relatively small, we can differentiate analytically. There also exist **automatic differentiation** software packages that can translate the source code of a program that computes $\mathbf{G}(\mathbf{m})$ into a program that computes the derivatives of $\mathbf{G}(\mathbf{m})$.

Another approach is to use **finite differences** to approximate the derivatives of $\mathbf{G}(\mathbf{m})_i$. A simple first-order scheme is to employ the linearization

$$\frac{\partial G(\mathbf{m})_i}{\partial m_j} \approx \frac{G(\mathbf{m} + h\mathbf{e}_j)_i - G(\mathbf{m})_i}{h}, \quad (9.49)$$

where $\mathbf{e}_j$ is the j th standard basis vector, and the scalar h is chosen to be suitably small. However, if h becomes very small, significant round-off error in the numerator of (9.49) may occur. A good rule of thumb is to set $h = \sqrt{\epsilon}$, where ϵ is the relative accuracy of the evaluations of $\mathbf{G}(\mathbf{m})_i$. For example, if the function evaluations are accurate to 0.0001, then an appropriate choice of h would be about 0.01. Determining the actual accuracy of function evaluations can be difficult, especially when $\mathbf{G}$ is a black box routine. One useful assessment technique is to plot function values as a parameter of interest is varied over a small range. These plots should be smooth at the scale of h . When $\mathbf{G}$ is available only as a black box subroutine that can be called with particular values of $\mathbf{m}$, and the source code for the subroutine is not available, then the only available approach is to use finite differences.

In practice, many difficulties in solving nonlinear regression problems can be traced back to incorrect derivative computations. It is thus a good idea to cross-check any available analytical formulas for the derivative with finite-difference approximations. Many software packages for nonlinear regression include options for checking the accuracy of derivative formulas.

A second important issue in the implementation of the GN and LM methods is deciding when to terminate the iterations. We would like to stop when the gradient $\nabla f(\mathbf{m})$ is approximately $\mathbf{0}$ and $\mathbf{m}$ has stopped changing substantially from one iteration to the next. Because of scaling issues, it is not possible to set an absolute tolerance on $\|\nabla f(\mathbf{m})\|_2$ that would be appropriate for all problems. Similarly, it is difficult to pick a single absolute tolerance on $\|\mathbf{m}^{k+1} - \mathbf{m}^k\|_2$ or $|f(\mathbf{m}^{k+1}) - f(\mathbf{m}^k)|$.

The following convergence tests have been normalized so that they will work well on a wide variety of problems. We assume that values of $\mathbf{G}(\mathbf{m})$ can be calculated with a relative accuracy of ϵ . To ensure that the gradient of $f(\mathbf{m})$ is approximately $\mathbf{0}$, we require that

$$\|\nabla f(\mathbf{m}^k)\|_2 < \sqrt{\epsilon}(1 + |f(\mathbf{m}^k)|). \quad (9.50)$$

To ensure that successive values of $\mathbf{m}$ are close, we require

$$\|\mathbf{m}^k - \mathbf{m}^{k-1}\|_2 < \sqrt{\epsilon}(1 + \|\mathbf{m}^k\|_2). \quad (9.51)$$

Finally, to make sure that the values of $f(\mathbf{m})$ have stopped changing, we require that

$$|f(\mathbf{m}^k) - f(\mathbf{m}^{k-1})| < \epsilon(1 + |f(\mathbf{m}^k)|). \quad (9.52)$$

There are a number of additional problems that can arise during the solution of a nonlinear regression problem by the GN or LM methods related to the functional behavior of $f(\mathbf{m})$.

The first issue is that our methods, because they involve calculation of the Jacobian and the Hessian, assume that $f(\mathbf{m})$ is a smooth function. This means not only that $f(\mathbf{m})$ must be continuous, but also that its first and second partial derivatives with respect to the parameters must be continuous. Figure 9.4 shows a function that is itself continuous, but has discontinuities in the first derivative at $m = 0.2$ and the second derivative at $m = 0.5$. When $\mathbf{G}(\mathbf{m})$ is given by an explicit formula, it is usually easy to verify that $f(\mathbf{m})$ is smooth, but when $\mathbf{G}(\mathbf{m})$ is implemented as a black box routine it may be difficult.

A second issue is that the function $f(\mathbf{m})$ may have a “flat bottom.” See Figure 9.5. In such cases, there are many values of $\mathbf{m}$ that come close to fitting the data, and it is difficult to determine the optimal $\mathbf{m}^*$. In practice, this condition is seen to occur when $\mathbf{J}(\mathbf{m}^*)^T \mathbf{J}(\mathbf{m}^*)$ is nearly singular. Because of this ill-conditioning, computing accurate confidence intervals for the model parameters can be effectively impossible. We will address this difficulty in Chapter 10 by applying regularization.

The final problem that we will consider is that $f(\mathbf{m})$ may be nonconvex and therefore have multiple local minimum points. See Figure 9.6. The GN and LM methods are designed to converge to a local minimum, but depending on where we begin the search, there is no way to be certain that such a solution will be a global minimum. Depending on the particular problem, the optimization algorithm might well converge to a locally optimal solution.

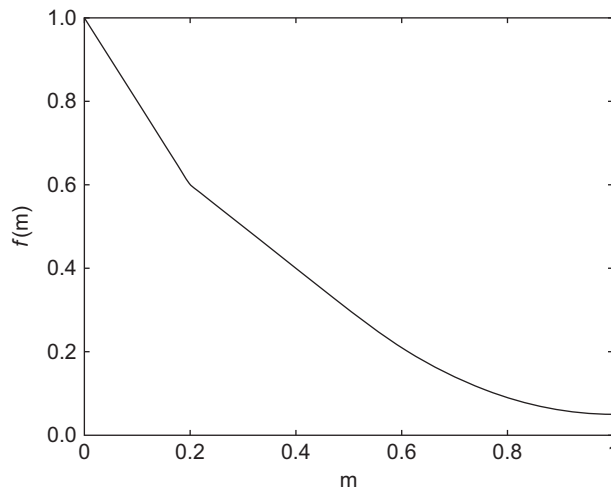


Figure 9.4 Example of a nonsmooth function.

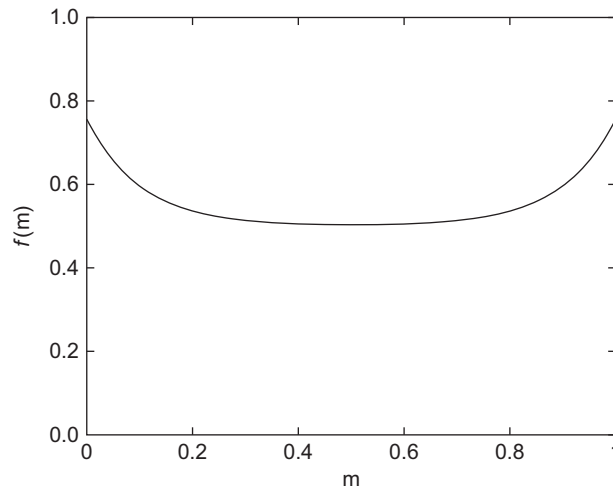


Figure 9.5 Example of a function with a flat bottom.

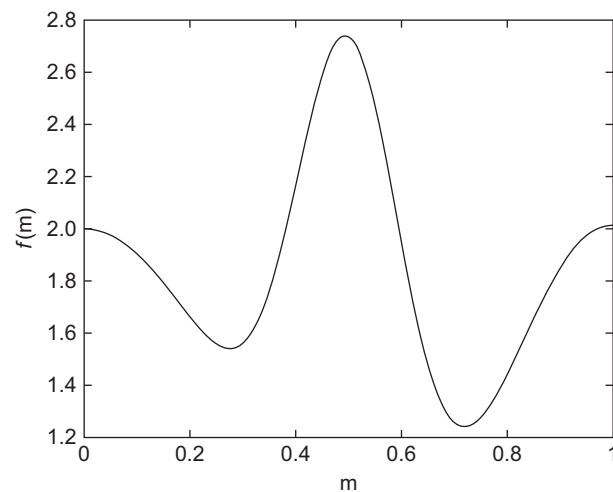


Figure 9.6 An example of a function with multiple local minima.

Global optimization methods have been developed to deal with this issue [17, 75, 76, 140]. Deterministic global optimization procedures can be used on problems with a very small number of variables, while stochastic search procedures can be applied to large-scale problems. Stochastic search procedures can be quite effective in practice, even though they do not find a global optimum with certainty.

However, even a deterministic global optimization procedure is not a panacea. In the context of nonlinear regression, if the nonlinear least squares problem has multiple locally

optimal solutions with similar objective function values, then each of these solutions will correspond to a statistically likely solution. We cannot simply report one globally optimal solution as our best estimate and construct confidence intervals using (9.38), because this would mean ignoring other likely solutions. However, if we could show that there is one globally optimal solution and other locally optimal solutions have very small p -values, then it would be appropriate to report the globally optimal solution and corresponding confidence intervals.

Although a thorough discussion of global optimization is beyond the scope of this book, we will discuss one simple global optimization procedure called the **multistart method** in the next example. In the multistart method, we randomly generate a large number of initial solutions, and then apply the LM method starting with each of these initial solutions. We then examine the local minimum solutions found by the procedure, and examine ones with acceptable values of $f(\mathbf{m})$. The multistart approach has two important practical advantages. First, by potentially identifying multiple locally optimal solutions, we can determine whether there is more than one statistically likely solution. Second, we can make effective use of the fast convergence of the LM method to find the locally optimal solutions.

Example 9.2

Consider the problem of fitting a model of two superimposed exponential decay functions characterized by four parameters,

$$y_i = m_1 e^{m_2 x_i} + m_3 x_i e^{m_4 x_i}, \quad (9.53)$$

to a set of observations. The true model parameters are $m_1 = 1.0$, $m_2 = -0.5$, $m_3 = 1.0$, and $m_4 = -0.75$, and the x_i values are 25 evenly spaced points between 1 and 7. We compute corresponding y_i values and add independent normally distributed noise with a standard deviation of 0.01 to obtain a synthetic data set.

We next apply the LM method to solve the problem 20 times, using random initial solutions, with each initial parameter uniformly distributed between -1 and 1 . This produces a total of three different locally optimal solutions (Table 9.2).

Solution number 1, with a χ^2 value of approximately 17, has an acceptable p -value of about 0.69 for a regression with 21 degrees of freedom. The other two solutions have



Table 9.2 Locally Optimal Solutions for the Sample Problem

Solution Number	m_1	m_2	m_3	m_4	χ^2	p -value
1	0.9874	-0.5689	1.0477	-0.7181	17.3871	0.687
2	1.4368	0.1249	-0.5398	-0.0167	40.0649	0.007
3	1.5529	-0.1924	-0.1974	-0.1924	94.7845	2×10^{-11}

unreasonably large χ^2 values, and hence much lower p -values. We will thus analyze only the first solution. [Figure 9.7](#) shows the data points with 1σ error bars and the fitted curve for solution number 1, and [Figure 9.8](#) shows the corresponding residuals normalized by the data standard deviations. Note that the majority of the residuals are within 0.5 standard deviations, with a few residuals as large as 1.9 standard deviations. There is no obvious residual trend as x ranges from 1 to 7.

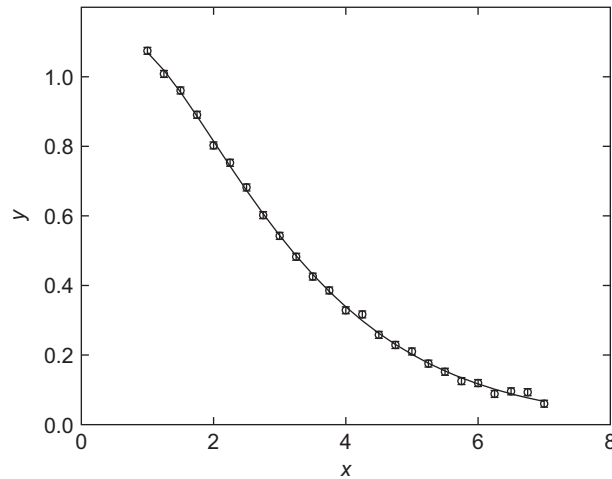


Figure 9.7 Data points and fitted curve.

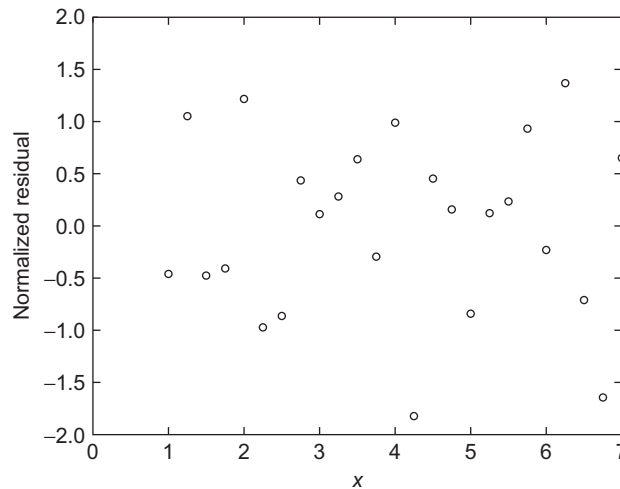


Figure 9.8 Normalized residuals corresponding to [Figure 9.7](#).

Next, we compute the approximate covariance matrix for the model parameters using (9.38). The square roots of the diagonal elements of the covariance matrix are standard deviations for the individual model parameters. These are then used to compute 95% confidence intervals for model parameters. The solution parameters with 95% confidence intervals are

$$\begin{aligned} m_1 &= 0.98 \pm 0.22 \\ m_2 &= -0.57 \pm 0.77 \\ m_3 &= 1.05 \pm 0.50 \\ m_4 &= -0.72 \pm 0.20. \end{aligned} \tag{9.54}$$

The true parameters (1, -0.5 , 1, and -0.75) are all covered by these confidence intervals. However, there is a large degree of uncertainty. This is an example of a poorly conditioned nonlinear regression problem in which the data do not strongly constrain the parameter values.

The correlation matrix provides some insight into the nature of the ill-conditioning in this example by quantifying strong parameter trade-offs. For our preferred solution, the correlation matrix calculated using (9.38) is

$$\rho = \begin{bmatrix} 1.00 & -0.84 & 0.68 & 0.89 \\ -0.84 & 1.00 & -0.96 & -0.99 \\ 0.68 & -0.96 & 1.00 & 0.93 \\ 0.89 & -0.99 & 0.93 & 1.00 \end{bmatrix}. \tag{9.55}$$

Note the strong positive and negative correlation between pairs of parameters. The high negative correlation between m_1 and m_2 tells us that by increasing m_1 and simultaneously decreasing m_2 we can obtain a solution that is very nearly as good as our optimal solution. There are also strong negative correlations between m_2 and m_3 and between m_2 and m_4 .

9.6. EXERCISES

1. Show that (9.29) is equivalent to the normal equations (2.3) when $G(\mathbf{m}) = \mathbf{Gm}$, where $\mathbf{G}$ is a matrix of constant coefficients.
2. A recording instrument sampling at 50 Hz records a noisy sinusoidal voltage signal in a 40-s-long record. The data are to be modeled using

$$y(t) = A \sin(2\pi f_0 t + \phi) + c + \sigma \eta(t) \tag{9.56}$$

where $\eta(t)$ is believed to be unit standard deviation, independent, and normally distributed noise, and σ is an unknown standard deviation. Using the data in the MATLAB data file **instdata.mat**, solve for the parameter (A, f_0, ϕ, c), using the



LM method. Show that it is critical to choose a good initial solution (suitable initial parameters can be found by examining a plot of the time series by eye). Once you are satisfied that you have found a good solution, use it to estimate the noise amplitude σ . Use your solution and estimate of σ to find corresponding covariance and correlation matrices and 95% parameter confidence intervals. Which pair of parameters is most strongly correlated? Are there multiple equally good solutions for this problem?

3. In hydrology, the van Genuchten model is often used to relate the volumetric water content in an unsaturated soil to the head [52]. The model is

$$\theta(h) = \theta_r + \frac{\theta_s - \theta_r}{(1 + (-\alpha h)^n)^{(1-1/n)}} \quad (9.57)$$

where θ_s is the volumetric water content at saturation, θ_r is the residual volumetric water content at a very large negative head, and α and n are two parameters which can be fit to laboratory measurements.



The file **vgdata.mat** contains measurements for a loam soil at the Bosque del Apache National Wildlife Refuge in New Mexico [68]. Fit the van Genuchten model to the data. The volumetric water content at saturation is $\theta_s = 0.44$, and the residual water content is $\theta_r = 0.09$. You may assume that the measurements of $\theta(h)$ are accurate to about 2% of the measured values.

You will need to determine appropriate values for σ_i , write functions to compute $\theta(h)$ and its derivatives, and then use the LM method to estimate the parameters. In doing so, you should consider whether or not this problem might have local minima. It will be helpful to know that typical values of α range from about 0.001 to 0.02, and typical values of n run from 1 to 10.

4. An alternative version of the LM method stabilizes the GN method by multiplicative damping. Instead of adding $\lambda \mathbf{I}$ to the diagonal of $\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k)$, this method multiplies the diagonal of $\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k)$ by a factor of $(1 + \lambda)$. Show that this method can fail by producing an example in which the modified $\mathbf{J}(\mathbf{m}^k)^T \mathbf{J}(\mathbf{m}^k)$ matrix is singular, no matter how large λ becomes.
5. A cluster of 10 small earthquakes occurs in a shallow geothermal reservoir. The field is instrumented with nine seismometers, eight of which are at the surface and one of which is 300 m down a borehole. The P-wave velocity of the fractured granite medium is thought to be an approximately uniform 2 km/s. The station locations (in meters relative to a central origin) are given in Table 9.3 and in the MATLAB data file **stmat.mat**.



The arrival times of P-waves from the earthquakes are carefully measured at the stations, with an estimated error of approximately 1 ms. The arrival time estimates for each earthquake, e_i , at each station (in seconds relative to an arbitrary reference), are given in Table 9.4. These data can also be found in the MATLAB data file **eqdata.mat**.

Table 9.3 Station Locations for the Earthquake Location Problem

Station	$x(m)$	$y(m)$	$z(m)$
1	500	-500	0
2	-500	-500	0
3	100	100	0
4	-100	0	0
5	0	100	0
6	0	-100	0
7	0	-50	0
8	0	200	0
9	10	50	-300

Table 9.4 Data for the Earthquake Location Problem.
Times are in Seconds

Station	e_1	e_2	e_3	e_4	e_5
1	0.8423	1.2729	0.8164	1.1745	1.1954
2	0.8680	1.2970	0.8429	1.2009	1.2238
3	0.5826	1.0095	0.5524	0.9177	0.9326
4	0.5975	1.0274	0.5677	0.9312	0.9496
5	0.5802	1.0093	0.5484	0.9145	0.9313
6	0.5988	1.0263	0.5693	0.9316	0.9480
7	0.5857	1.0141	0.5563	0.9195	0.9351
8	0.6017	1.0319	0.5748	0.9362	0.9555
9	0.5266	0.9553	0.5118	0.8533	0.8870

Station	e_6	e_7	e_8	e_9	e_{10}
1	0.5361	0.7633	0.8865	1.0838	0.9413
2	0.5640	0.7878	0.9120	1.1114	0.9654
3	0.2812	0.5078	0.6154	0.8164	0.6835
4	0.2953	0.5213	0.6360	0.8339	0.6982
5	0.2795	0.5045	0.6138	0.8144	0.6833
6	0.2967	0.5205	0.6347	0.8336	0.6958
7	0.2841	0.5095	0.6215	0.8211	0.6857
8	0.3025	0.5275	0.6394	0.8400	0.7020
9	0.2115	0.4448	0.5837	0.7792	0.6157

- a. Apply the LM method to this data set to estimate least squares locations of the earthquakes.
- b. Estimate the uncertainties in x , y , z (in meters) and origin time (in seconds) for each earthquake using the diagonal elements of the appropriate covariance matrix. Do the earthquake locations follow any discernible trend?

Table 9.5 Data for the Lightning Mapping Array Problem

Station	t(s)	x(km)	y(km)	z(km)
1	0.0922360280	-24.3471411	2.14673146	1.18923667
2	0.0921837940	-12.8746056	14.5005985	1.10808551
3	0.0922165500	16.0647214	-4.41975194	1.12675062
4	0.0921199690	0.450543748	30.0267473	1.06693166
6	0.0923199800	-17.3754105	-27.1991732	1.18526730
7	0.0922839580	-44.0424408	-4.95601205	1.13775547
8	0.0922030460	-34.6170855	17.4012873	1.14296361
9	0.0922797660	17.6625731	-24.1712580	1.09097830
10	0.0922497250	0.837203704	-10.7394229	1.18219520
11	0.0921672710	4.88218031	10.5960946	1.12031719
12	0.0921702350	16.9664920	9.64835135	1.09399160
13	0.0922357370	32.6468622	-13.2199767	1.01175261

6. The Lightning Mapping Array is a portable system that locates the sources of lightning radio-frequency radiation in three spatial dimensions and time [132]. The system measures the arrival times of impulsive radiation events with an uncertainty specified by a standard deviation of $7 \times 10^{-2} \mu\text{s}$. Measurements are made at multiple locations, typically in a region 40 to 60 km in diameter. Each station records the peak radiation event in successive 100- μs time intervals; from such data, several hundred to over a thousand distinct radiation sources may typically be located per lightning discharge. Example data from the LMA are shown in Table 9.5 and are found in the MATLAB data file **lightningdata.mat**.



- Use the arrival times at stations 1, 2, 4, 6, 7, 8, 10, and 13 to find the time and location of the associated source. Assume that radio wave signals travel along straight paths at the speed of light in a vacuum ($2.997 \times 10^8 \text{ m/s}$).
- A challenge in dealing with the large number of detections from the LMA is to disentangle overlapping signals from multiple events. Locate using subsets of the data set to find the largest subset of the data for nine or more stations that gives a good solution, and compare it to the station subset from part (a).

9.7. NOTES AND FURTHER READING

Newton's method is central to the field of optimization [39, 58, 87, 88, 119], and is, because of its speed, the basis for most methods of nonlinear optimization. A number of modifications to the method are used to ensure convergence to a local minimum of $f(\mathbf{x})$ [58, 119]. One important difficulty in Newton's method is that, for very large problems, it may be impractical to store the Hessian matrix. Specialized methods have been developed for the solution of such large-scale optimization problems [58, 119].

The GN and LM methods are discussed in more detail in [16, 58, 119]. Statistical aspects of nonlinear regression are discussed in [7, 40, 113]. A more detailed discussion of the termination criteria for the LM method described in [Section 9.5](#) can be found in [58]. There are a number of freely available and commercial software packages for nonlinear regression, including GaussFit [82], MINPACK [111], and ODRPACK [18]. Automatic differentiation has applications in many areas of numerical computing, including optimization and numerical solution of ordinary and partial differential equations. Two books that survey this topic are [34, 57]. Global optimization is a large field of research. Some basic references include [17, 75, 76]. For a survey of global optimization methods in geophysical inversion, see [140].